

KERN-LITE

Embedded Black-Box Data Logger

Student Project Specification

STM32L4 | CMSIS-RTOS2 / FreeRTOS | FatFs | Python Ground Station

Build a fault-aware embedded recorder that samples and filters sensor data, protects it with CRC-32, stores it in a recoverable circular file ring, and exposes live and historical telemetry to a Python operator console.

DOCUMENT ROLE This specification defines what the completed system must do. The Complete Student Project Handbook defines the setup procedure, phase order, implementation tasks, and daily acceptance gates.

Protocol and record layouts are fixed. Deliberate deviations require instructor approval and written documentation.

1. Document Purpose and Authority

This document defines the mandatory functional behavior, interfaces, data layouts, architectural constraints, validation evidence, and submission requirements for KERN-LITE. It is a requirements document, not a daily implementation plan.

CANONICAL RELATIONSHIP Use this specification to determine what must be delivered. Use the Complete Student Project Handbook to determine how the team progresses through Phase 0 and Days 1-7. The two documents are aligned.

1.1 Normative language

Term	Meaning
shall	Mandatory requirement. The completed project is expected to satisfy it.
should	Recommended engineering practice. A different approach must be justified.
may	Optional behavior or implementation choice.
out of scope	Must not be required for project completion or grading.

1.2 Scope boundary

- In scope: STM32L4 firmware, CMSIS-RTOS2/FreeRTOS tasking, sensor processing, CRC-protected binary UART, circular FatFs storage, Python ground station, analytics, persistence, export, and recovery testing.
- Out of scope: the retired event-slot/event-dump system, dynamic event metadata, cloud services, networking beyond the local serial connection, multi-device ground-station support, and production security features.
- The trigger/latch input may be initialized for hardware exercises, but it is not a stored telemetry channel and no firmware event-log subsystem is required.

2. Specification Map

The following map provides a compact view of the document and the relationship between major requirement groups.

Section	Content
3	Project Overview and Learning Objectives
4	System Context and Architecture
5	Functional Requirements
6	Firmware Architecture and Responsibilities
7	Ground Station Requirements
8	Communication Protocol
9	Sensor Processing and Record Format
10	Circular File Storage
11	CRC and Integrity Rules
12	State Machine and Operational Behavior
13	RTOS, Concurrency, and Resource Constraints
14	Verification and Acceptance
15	Deliverables and Submission
16	Grading Considerations
A	Fixed Constants and Defaults
B	Glossary

3. Project Overview and Learning Objectives

KERN-LITE is a miniature embedded black-box recorder. The firmware samples five scalar telemetry channels, filters and validates the data, builds a fixed 32-byte record, writes records into a circular set of SD-card files, and streams telemetry over a strict binary protocol. A Python Ground Station controls the recorder, verifies integrity, visualizes live and replayed data, persists sessions, and exports evidence.

3.1 Central engineering theme

DATA SURVIVABILITY Every stored record is independently checksummed, the storage ring overwrites only the oldest retained data, metadata is periodically persisted, and the recorder reconstructs its write head after an unexpected reset.

3.2 Learning objectives

- Design a robust binary framing protocol with CRC integrity and deterministic little-endian layouts.
- Implement a byte-at-a-time decoder that survives noise, malformed lengths, truncation, corruption, and resynchronization.
- Build a statically allocated four-task RTOS application with explicit ownership and synchronization.
- Apply moving-average filtering, validation, thresholds, and hysteresis to sampled data.
- Implement a fixed-capacity multi-file ring with overwrite, replay, metadata persistence, and crash recovery.
- Prove bit-exact CRC agreement between independent C++ and Python implementations.
- Write host-side unit tests for logic that does not require STM32 hardware.
- Build a Python operator console with telemetry models, charts, statistics, integrity reporting, sessions, and export.

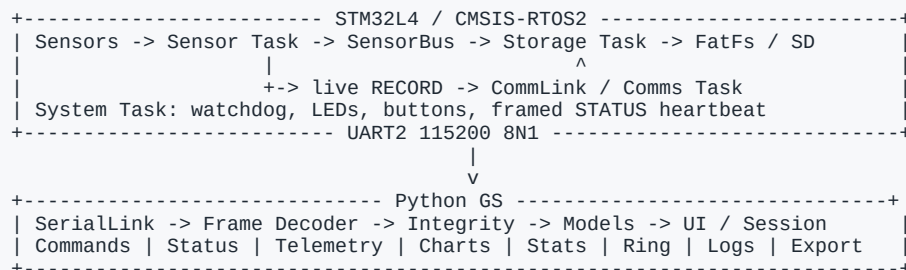
3.3 Telemetry channels

Channel	Source	Nominal update	Stored representation
LM35 temperature	LM35 analog sensor	10 Hz	int16, degrees C x10
DHT temperature	DHT11 digital sensor	New measurement about every 2 s; last good value reused at 10 Hz	int16, degrees C x10
DHT humidity	DHT11 digital sensor	New measurement about every 2 s; last good value reused at 10 Hz	uint16, %RH x10
Light	Photodiode analog input	10 Hz	uint16 normalized x65535
Potentiometer	Analog potentiometer	10 Hz	uint16 normalized x65535

Buttons and the optional trigger/latch input are control inputs, not continuous stored telemetry channels. The retired event system is not part of KERN-LITE.

4. System Context and Architecture

The project contains two independently testable systems joined by one binary serial interface: the embedded recorder and the host Ground Station.



4.1 Mandatory firmware layering

Layer	Responsibility	Host-testable?
firmware/hal	GPIO, ADC, UART-facing adapters, watchdog, timer/PWM/tone, EXTI helpers	No; board-specific

Layer	Responsibility	Host-testable?
firmware/sensors	Small driver per physical sensor	Drivers may use mocked HAL
firmware/dsp	Moving average, channel processing, threshold detector with hysteresis	Yes
firmware/protocol	Frame definitions, CRC wrapper, encoder, byte decoder	Yes
firmware/storage	SensorRecord and CircularLog	Index/recovery logic yes; FatFs via shim
firmware/recorder	StateMachine, CommandHandler, CommLink, SensorBus	State/handler logic yes
firmware/system	Orchestrator, task entry points, configuration, ownership	Mostly target-side
groundstation	Link, protocol mirror, state/telemetry models, analytics, sessions, UI, export	Yes with pytest

SEPARATION RULE Protocol, DSP, state, record-layout, and ring-index logic shall not depend on STM32 headers. Hardware adapters shall not contain application policy.

5. Functional Requirements

5.1 Embedded recorder requirements

ID	Requirement
FR-FW-01	On cold boot, the firmware shall initialize hardware, mount or recover storage, start the scheduler, and enter Idle.
FR-FW-02	While Recording, the Sensor task shall produce one SensorRecord every 100 ms nominally.
FR-FW-03	Each SensorRecord shall contain timestamp, millisecond part, monotonic 16-bit sequence, five scaled telemetry values, alert bits, state, fault bits, reserved bytes, and record CRC.
FR-FW-04	The analog channels shall pass through moving-average filtering before scaling and threshold evaluation.
FR-FW-05	Threshold alerts shall use hysteresis so values near a boundary do not chatter.
FR-FW-06	Sensor faults shall be non-fatal: set fault_bits, preserve the last known good value, and continue recording.
FR-FW-07	Every record written to storage shall carry a valid record-level CRC-32 over bytes 0 through 27.
FR-FW-08	While Recording, each new record shall be appended to the circular file ring and streamed as a RECORD frame.
FR-FW-09	The firmware shall implement START, STOP, STATUS, REPLAY, and ERASE according to the state and payload rules in Section 8.
FR-FW-10	REPLAY N shall return the newest retained N records in chronological order and then send ACK.
FR-FW-11	The firmware shall reject malformed or CRC-invalid incoming frames without dispatching their command.
FR-FW-12	The firmware shall persist storage metadata every 16 records and on every controlled stop.
FR-FW-13	After an unexpected reset, the firmware shall recover the newest valid write position without damaging previously valid records.
FR-FW-14	The firmware shall tolerate transient SD write failures and enter Fault only after three consecutive failures.
FR-FW-15	While connected operation is expected, the firmware shall transmit a valid framed STATUS at least every 5 seconds as the protocol heartbeat.
FR-FW-16	From Day 2 onward, UART2 shall contain framed binary protocol data only. Raw diagnostic ASCII is prohibited on the protocol UART.

ID	Requirement
FR-FW-17	ERASE shall require Idle state and the exact magic value 0xDEADC0DE.
FR-FW-18	The watchdog shall be refreshed by the System task and shall reset the device if supervision stalls beyond the configured timeout.

5.2 Ground Station requirements

ID	Requirement
FR-GS-01	The GS shall connect and disconnect through a selectable serial port at 115200 baud by default.
FR-GS-02	The link manager shall decode byte streams incrementally and maintain RX, TX, CRC-error, sync-error, NACK, reconnect, and latency metrics.
FR-GS-03	The GS shall auto-reconnect after a link drop without resetting accumulated link counters.
FR-GS-04	The GS shall decode STATUS into Idle, Recording, or Fault and gate command controls accordingly.
FR-GS-05	The command panel shall support START, STOP, STATUS, REPLAY N, and ERASE with typed magic confirmation.
FR-GS-06	The GS shall validate the frame CRC before decoding any frame payload.
FR-GS-07	For each RECORD, the GS shall independently validate the record CRC and flag storage corruption when frame CRC passes but record CRC fails.
FR-GS-08	The GS shall decode and display current values for all five telemetry channels with units and alert indication.
FR-GS-09	The GS shall maintain incremental min, max, mean, sample standard deviation, alert activations, time in alert, and percent in alert per channel.
FR-GS-10	The GS shall provide a rolling chart with channel toggles, thresholds, alert shading, state markers, reboot markers, and sequence-gap markers.
FR-GS-11	The GS shall display SD-mounted status, file count, active file, write index, cumulative records, ring wraps, live record count, and replay record count.
FR-GS-12	The GS shall maintain a filterable timestamped history of alerts, clears, state transitions, NACKs, integrity errors, sequence gaps, reconnects, heartbeat timeouts, and GS actions.
FR-GS-13	The GS shall detect probable reboots from sequence reset and STATUS changes, log the event, and re-issue STATUS.
FR-GS-14	A new timestamped session shall be created on connect and continuously store record bytes, wall-clock timestamps, and raw TX/RX frame logs.
FR-GS-15	Previous sessions shall be reloadable so telemetry, statistics, logs, and timeline can be reconstructed.
FR-GS-16	The GS shall export session records to CSV and alert, raw-frame, and timeline information to text.
FR-GS-17	The GS shall compute a 0-100 link-quality score from CRC errors, sync errors, gaps, NACKs, and timing jitter.
FR-GS-18	The GS shall log HEARTBEAT_TIMEOUT if no valid STATUS or RECORD frame is received for 15 seconds while connected.

5.3 Non-functional requirements

ID	Requirement
NFR-01	The firmware shall use CMSIS-RTOS2 over FreeRTOS and static task allocation.
NFR-02	No heap allocation is permitted in steady-state firmware operation.

ID	Requirement
NFR-03	One Orchestrator shall own all long-lived application objects.
NFR-04	ISR code shall be bounded, non-blocking, and shall not allocate memory.
NFR-05	Shared transmit access shall protect the complete frame encode-and-transmit sequence.
NFR-06	The project shall build without warnings; warnings are treated as errors.
NFR-07	Protocol, record, metadata, and disk fields shall use little-endian encoding.
NFR-08	Pure logic shall have repeatable host-side unit tests independent of hardware.
NFR-09	The repository shall be reproducible from documented build and run instructions.
NFR-10	Fault handling shall preserve valid historical data whenever possible.
NFR-11	Task communication shall use explicit interfaces rather than cross-layer access.
NFR-12	The default 4 x 256 record ring shall wrap in about 102.4 seconds at 10 Hz for lab demonstration.

6. Firmware Architecture and Responsibilities

6.1 Task model

Task	Nominal cadence	Priority intent	Responsibilities
Sensor	100 ms	Medium	Read channels; update DHT periodically; validate/filter; compute alert/fault bits; build and publish record; initiate live stream.
Storage	Event/100 ms loop	Medium	Consume the latest unpublished record; write to CircularLog; track consecutive failures; trigger SD fault policy.
Comms	10 ms poll plus UART ISR feed	High	Poll complete decoded frames; dispatch commands; transmit responses and replay records.
System	50 ms	Low	Kick watchdog; drive LEDs/tone; poll buttons; handle remount/recovery; send periodic framed STATUS.

6.2 Ownership and communication

- Orchestrator owns the state machine, sensor objects, DSP channels, SensorBus, CircularLog, CommLink, CommandHandler, and supervision state.
- Sensor-to-storage transfer shall prevent duplicate writes. A simple latest-value getter is insufficient unless accompanied by generation or sequence tracking.
- UART RX ISR feeds one byte to CommLink/Decoder and immediately re-arms reception. Complete frames are transferred safely to the Comms task.
- CommLink transmission shall serialize live records, command responses, and replay traffic using one static mutex.
- No task shall directly manipulate another layer's internal state.

6.3 Status indicators

Condition	Required indication
Idle	Recording LED off; system remains responsive; framed STATUS heartbeat continues.
Recording, nominal	Solid green or course-defined nominal recording indication.
Recording with sensor fault bits	Blinking green or equivalent degraded-but-recording indication.
Fault	Blinking red and fault tone pattern.

Condition	Required indication
Phase 0 only	PC9 heartbeat toggle and ASCII KERN-LITE ALIVE smoke output are permitted before the binary protocol is activated.

7. Ground Station Architecture and Responsibilities

7.1 Recommended module decomposition

Module	Primary responsibility
link.py	Serial connection, decoder feed, reconnect, counters, latency, receive dispatch.
frame.py / crc.py	Protocol constants, enums, Frame, encoder, decoder, CRC-32.
commands.py	Command construction, state-aware send functions, STATUS poller.
state.py	Device state, transition history, command gating, reboot indicators.
telemetry.py	SensorRecord decode, scaling, telemetry session model.
integrity.py	Record-level CRC and sequence-gap checks.
storage_panel.py	STATUS storage model and ring visualization.
session.py	Session binary storage, raw frame log, load/reconstruct.
chart.py / stats.py	Rolling channel data and incremental statistics.
timeline.py / alert_log.py	State bands, alert track, event history, text export.
link_quality.py	Quality score, jitter and heartbeat monitoring.
export.py	CSV and human-readable text exports.

7.2 Live versus replay traffic

The RECORD frame type is used for both live and replayed records. The GS shall classify incoming records using command context: records arriving during an active REPLAY transaction increment the replay counter; normal Recording-state records increment the live counter. The transaction ends when ACK is received.

INTEGRITY DISTINCTION Frame CRC protects serial transport. Record CRC protects the stored record. A replayed record with an invalid record CRC is still framed with a valid frame CRC so the GS can report storage corruption.

8. Communication Protocol

8.1 Physical link

Property	Value
Peripheral	UART2
Format	115200 baud, 8 data bits, no parity, 1 stop bit, no flow control
Byte order	Little-endian for all multi-byte fields
Receive mode	Byte-at-a-time interrupt feeding a stateful decoder
Post-setup wire content	Binary frames only; no raw debug text after Day 2

8.2 Frame layout

```
[STX 0xAB][TYPE u8][LEN u16 LE][PAYLOAD 0..256][CRC32 u32 LE][ETX 0xCD]
```

Field	Size	CRC coverage	Notes
STX	1	No	0xAB
TYPE	1	Yes	Opcode
LEN	2	Yes	Payload length, 0..256
PAYLOAD	LEN	Yes	May be empty
CRC32	4	No	CRC over TYPE + LEN bytes + PAYLOAD
ETX	1	No	0xCD

CORRECT FRAME SIZE Fixed frame overhead is 9 bytes, not 8. A zero-payload ACK frame is 9 bytes; the maximum encoded frame size is 265 bytes.

8.3 Frame types

Direction	Name	Type	Payload
GS -> KERN	CMD_START	0x01	None
GS -> KERN	CMD_STOP	0x02	None
GS -> KERN	CMD_STATUS	0x03	None
GS -> KERN	CMD_REPLAY	0x04	u16 N LE; default 120 if omitted
GS -> KERN	CMD_ERASE	0x06	u32 magic LE = 0xDEADC0DE
KERN -> GS	RECORD	0x10	32-byte SensorRecord
KERN -> GS	STATUS	0x12	14-byte status block
KERN -> GS	ACK	0x20	None
KERN -> GS	NACK	0x21	u8 NackCode

Opcodes 0x05, 0x07, and 0x11 are retired and shall not be used.

8.4 NACK codes

Code	Name	Meaning
0x01	CrcError	Reserved for a command whose corruption can be identified; ordinary frame-CRC failures are dropped before dispatch.
0x02	BadCommand	Unknown or retired opcode.
0x03	InvalidState	Command is not legal in the current state.
0x04	StorageError	Storage is unavailable or the operation failed.
0x06	BadMagic	ERASE payload is missing or incorrect.

8.5 STATUS payload

Offset	Field	Type	Meaning
0	state	u8	0 Idle, 1 Recording, 2 Fault
1	sd_mounted	u8	0 or 1
2	file_count	u8	Configured log-file count
3	current_file	u8	Active file index
4..7	total_records	u32 LE	Cumulative records written since last erase/reinitialize; may exceed ring capacity
8..11	wrap_count	u32 LE	Completed full-ring wraps
12..13	records_in_file	u16 LE	Current write index within active

Offset	Field	Type	Meaning
			file

8.6 Command validity and response behavior

Command	Valid state	Success behavior	Failure
START	Idle only	Transition to Recording; send STATUS; send ACK.	NACK InvalidState
STOP	Recording only	Flush metadata; transition to Idle; send STATUS; send ACK.	NACK InvalidState or StorageError
STATUS	Any	Send one STATUS frame.	No state failure
REPLAY	Idle only and SD mounted	Stream newest retained N in chronological order; then ACK.	NACK InvalidState / StorageError
ERASE	Idle only	Validate magic; erase and reinitialize ring; send ACK.	NACK InvalidState / BadMagic / StorageError

A frame that fails frame CRC, declares LEN > 256, or fails end-marker validation shall never be dispatched.

8.7 Decoder behavior

- Required states: WaitStx, Type, LenLo, LenHi, Payload, Crc0, Crc1, Crc2, Crc3, WaitEtx.
- CRC shall be accumulated incrementally over TYPE, both LEN bytes, and every payload byte.
- LEN greater than 256 shall cause SyncError and reset without writing beyond the payload buffer.
- Garbage before STX shall be ignored. After error, the decoder shall return to a state that can accept the next valid STX.
- A complete valid frame shall be copied into task-owned pending storage before the decoder reuses its internal buffer.

9. Sensor Processing and Record Format

9.1 Signal processing

Channel	Low	High	Hysteresis
LM35	10 deg C	40 deg C	2 deg C
DHT temperature	5 deg C	45 deg C	2 deg C
DHT humidity	10 %RH	90 %RH	5 %RH
Light	0.05	0.95	0.05
Potentiometer	0.02	0.98	0.02

- Analog channels shall use a moving-average window of 16 samples by default.
- A HighAlert clears only after the value falls below high minus hysteresis.
- A LowAlert clears only after the value rises above low plus hysteresis.
- DHT11 measurements shall be requested no more often than the sensor supports; the last valid reading is inserted into each 10 Hz record.
- A failed DHT measurement sets the appropriate fault flag and retains the prior valid value.

9.2 Fixed SensorRecord layout

```
#pragma pack(push, 1)
struct SensorRecord {
    uint32_t timestamp; // uptime seconds
    uint16_t ms;        // 0..999
    uint16_t seq;        // monotonic, modulo 65536
    int16_t  lm35_c;      // deg C x10
    int16_t  dht_temp_c;  // deg C x10
    uint16_t dht_hum;     // %RH x10
    uint16_t light;       // normalized x65535
    uint16_t pot;         // normalized x65535
    uint8_t  alert_bits;
    uint8_t  state;
    uint8_t  fault_bits;
    uint8_t  reserved[7];
    uint32_t crc32;      // CRC over bytes 0..27
};
```

```
#pragma pack(pop)
static_assert(sizeof(SensorRecord) == 32);
static_assert(offsetof(SensorRecord, crc32) == 28);
```

9.3 Byte offsets

Offset	Size	Field	Encoding
0	4	timestamp	u32 LE
4	2	ms	u16 LE
6	2	seq	u16 LE
8	2	lm35_c	i16 LE
10	2	dht_temp_c	i16 LE
12	2	dht_hum	u16 LE
14	2	light	u16 LE
16	2	pot	u16 LE
18	1	alert_bits	bit mask
19	1	state	0/1/2
20	1	fault_bits	bit mask
21	7	reserved	zero when written
28	4	crc32	u32 LE over bytes 0..27

9.4 Alert and fault masks

Mask	alert_bits
0x01	LM35_HI
0x02	LM35_LO
0x04	LIGHT_HI
0x08	LIGHT_LO
0x10	POT_HI
0x20	POT_LO
0x40	DHT_TEMP_HI
0x80	DHT_HUM_HI

Mask	fault_bits
0x01	LM35_RANGE
0x02	DHT_TIMEOUT
0x04	DHT_BADDATA
0x08	LIGHT_STUCK
0x10	POT_STUCK

10. Circular File Storage

10.1 Capacity and files

Constant	Default	Effect
LOG_FILE_COUNT	4	LOG00.BIN through LOG03.BIN

Constant	Default	Effect
RECORDS_PER_FILE	256	8192 bytes per full file
Ring capacity	1024 records	32768 bytes of retained records
Sample rate	10 Hz	Full ring in about 102.4 seconds
META_FLUSH_EVERY_N	16	Metadata may trail writes; recovery must scan forward

10.2 Write behavior

- Records are written sequentially into `current_file` at `write_index`.
- After writing index 255, `write_index` resets to 0 and `current_file` advances modulo 4.
- Advancing from file 3 to file 0 increments `wrap_count` and begins overwriting the oldest retained records.
- `total_records` increments on every successful write and is not limited by ring capacity; ERASE resets it.
- The input `SensorRecord` shall be copied before its CRC field is updated so `writeRecord` does not mutate a const caller-owned object.
- A failed physical write shall not advance metadata or the logical write head.

10.3 Canonical metadata format

```
#pragma pack(push, 1)
struct LogMeta {
    uint32_t magic;           // 0x4C4F4700, "LOG\0"
    uint32_t version;         // 1
    uint8_t file_count;       // 4
    uint16_t records_per_file; // 256
    uint8_t current_file;
    uint16_t write_index;
    uint32_t wrap_count;
    uint32_t total_records;
    uint8_t reserved[10];     // pad to 32 bytes before CRC
    uint32_t crc32;           // CRC over bytes 0..31
};
#pragma pack(pop)
static_assert(sizeof(LogMeta) == 36);
```

Metadata rule	Requirement
Validation	magic, version, configured dimensions, field ranges, and CRC shall be checked.
Flush	Write all 36 bytes and call <code>f_sync</code> at least every 16 successful records and on STOP.
Initialization	If no valid record exists, initialize an empty ring and write valid metadata.
Erase	Delete all LOGxx.BIN and META.BIN, then remount/reinitialize only after validating erase magic.

10.4 Recovery

- If metadata is valid, treat it as a checkpoint, not necessarily the final head. Scan forward from the saved head to recover additional valid records written since the last flush.
- If metadata is missing or corrupt, scan all record slots and validate record CRCs.
- After a complete wrap every slot may contain a valid record, so a simple valid-to-invalid boundary is insufficient.
- Determine the newest valid record using timestamp, millisecond field, and modulo-aware sequence ordering. The next circular slot becomes the recovered write head.
- Recovery shall not rewrite or erase valid records.

10.5 Replay

- Clamp `N` to `min(requested N, total_records, ring capacity)`.
- Let `G = current_file * RECORDS_PER_FILE + write_index`. Compute `start = (G - N + capacity) % capacity`.
- Walk forward `N` positions from start and deliver records oldest-first.
- A stored record with a bad record CRC shall still be sent in a newly valid RECORD frame so the GS can flag storage corruption and continue with surrounding records.
- A file read failure aborts replay and produces `StorageError`; a record-level CRC mismatch alone does not abort replay.
- `N = 0` produces no RECORD frames and an immediate ACK.

10.6 Required interface

```
class CircularLog {
public:
    StorageStatus mount();
    StorageStatus writeRecord(const SensorRecord& r);
    StorageStatus replayNewest(uint32_t n, RecordCb cb, void* ctx);
    StorageStatus eraseAll(uint32_t magic);
    uint32_t totalRecords() const;
    uint32_t wrapCount() const;
    uint8_t currentFile() const;
    uint16_t writeIndex() const;
    bool isMounted() const;
};
```

11. CRC and Integrity Rules

Parameter	Value
Name	CRC-32 / IEEE 802.3 / ITU-T V.42
Normal polynomial	0x04C11DB7
Reflected polynomial	0xEDB88320
Initial value	0xFFFFFFFF
Input reflection	Yes
Output reflection	Yes
Final XOR	0xFFFFFFFF
Known-answer test	CRC32("123456789") = 0xCBF43926

11.1 CRC locations

Layer	Covered bytes	Failure behavior
Frame	TYPE + LEN_LO + LEN_HI + PAYLOAD	Drop/count frame; never dispatch command.
Record	SensorRecord bytes 0..27	GS flags storage corruption; replay continues if transport is valid.
Metadata	LogMeta bytes 0..31	Reject checkpoint and perform recovery scan.

DAY 1 BLOCKER The firmware and Python KAT shall both equal 0xCBF43926 before protocol integration continues.

12. State Machine and Operational Behavior

12.1 States

Code	State	Meaning
0	Idle	Storage may be mounted; commands accepted according to rules; no live sampling, writing, or streaming.
1	Recording	10 Hz records are built, written, and streamed.
2	Fault	Recording is suspended because persistent storage operation failed; recovery is attempted.

12.2 Transitions

From	Event	To	Required action
Boot	Initialization and storage check complete	Idle	Send STATUS; remain command-responsive.
Idle	Valid START	Recording	Enable record production; update indication; send STATUS then ACK.
Recording	Valid STOP	Idle	Flush metadata; stop record production; send STATUS then ACK.
Recording	Short press	Idle	Flush metadata and send STATUS.
Recording	Third consecutive SD write failure	Fault	Stop storage writes/live production; red indication; send STATUS.
Fault	Remount and recovery succeed	Recording	Resume at recovered head; send STATUS.
Fault	Repeated recovery failure	Fault/reboot	Apply course-defined retry limit, then allow watchdog or controlled reset.

12.3 Expected scenarios

Scenario	Expected behavior
Cold boot	Mount/recover ring, enter Idle, send periodic framed STATUS.
START	Transition Idle -> Recording; begin 10 Hz write and live stream.
File advance	write_index reaches 256; next file becomes current.
Ring wrap	File 3 advances to file 0; wrap_count increments; oldest data is overwritten.
STOP	Flush metadata, stop live traffic, return to Idle.
REPLAY	Idle only; send retained records oldest-of-N first; ACK terminates transaction.
Unexpected reset	Recover head from checkpoint plus scan or full scan; preserve valid records; GS flags reboot.
Sensor fault	Set fault bit and keep recording using last good data.
Persistent SD fault	After three consecutive failures enter Fault; remount/recover; resume when successful.
Bad frame	Count/drop; no command effect.
Bad stored record	Transport it during replay in a valid outer frame; GS reports corruption.

13. RTOS, Concurrency, and Resource Constraints

- CubeMX middleware interface shall be CMSIS_V2. Native FreeRTOS calls may be used inside the firmware where the handbook explicitly requires static allocation or synchronization.
- Four application tasks shall be created statically. Required Idle and Timer task memory callbacks shall be supplied when static allocation is enabled.
- No malloc/new shall be used during steady-state operation. Static buffers shall cover frame encoding, RX, task stacks, and synchronization objects.
- The UART ISR shall only capture/feed a byte, record bounded status, and re-arm RX. It shall not call blocking transmit, FatFs, printf, or application command logic.
- CommLink shall protect TX with a static mutex across both encode and HAL transmission.
- SensorBus shall guarantee that each published record is written no more than once and that a slow Storage task cannot read a partially updated record.
- Task delays shall use RTOS timing and shall not busy-wait except for tightly bounded hardware timing where unavoidable.
- The DHT transaction shall be bounded and shall not hold shared locks or disable interrupts for the full transaction.
- Watchdog refresh belongs to the System task so starvation or scheduler failure is detectable.
- Stack high-water marks shall be observed during validation and task stacks adjusted with evidence rather than guesswork.

13.1 Default task resources

Task	Stack words	Priority
Sensor	512	idle + 2
Storage	768	idle + 2
Comms	384	idle + 3
System	384	idle + 1

These are starting values from the handbook. Teams may increase a stack only after documenting the observed high-water mark and keeping static RAM within target limits.

14. Verification and Acceptance

14.1 Required host tests

Test area	Minimum evidence
CRC	Known-answer vector; incremental and one-shot agreement.
Codec	Zero/max payload, round trip, corruption, malformed length, garbage resynchronization, C++/Python cross-vectors.
DSP	Moving average outputs; high/low transitions; hysteresis; no chatter.
Record	Size 32; CRC offset 28; scaling and signed decode; record CRC.
Storage math	File advance, full wrap, overwrite, replay clamp, cross-boundary chronological order, N=0.
Metadata/recovery	Metadata CRC rejection; stale-checkpoint forward scan; full-ring newest-record recovery.
FSM/commands	Valid transitions and all InvalidState/BadMagic/StorageError cases.
GS models	STATUS decode, telemetry stats, state transitions, sessions, integrity, link quality, timeline exports.

14.2 Hardware and integration acceptance

ID	Requirement
AC-01	Phase 0 scheduler smoke test: clean build, heartbeat LED, UART smoke output.
AC-02	CMD_STATUS round-trip on hardware with correct 14-byte decode.
AC-03	Five consecutive live records with plausible values and both CRCs valid.
AC-04	Threshold alert sets and clears with visible hysteresis.
AC-05	Full 4 x 256 ring wrap demonstrated in about 110 seconds.
AC-06	REPLAY 300 crosses a file boundary and returns exactly 300 chronological records when available.
AC-07	REPLAY exceeding capacity clamps to retained count; REPLAY 0 returns immediate ACK.
AC-08	Power loss mid-write preserves prior valid records and recovers the write head.
AC-09	Invalid command/state/magic combinations produce the defined NACK.
AC-10	Corrupt serial frame increments error count without disturbing continued operation.
AC-11	Corrupt stored record produces a GS storage-corruption warning while surrounding records continue.
AC-12	USB-UART disconnect/reconnect restores operation and counters continue.
AC-13	DHT fault sets correct mask and last-known-good data remains visible.

ID	Requirement
AC-14	Three consecutive SD write failures enter Fault; successful remount returns to Recording.
AC-15	All host C++ and pytest suites pass from documented one-command entry points.

14.3 Acceptance evidence

- [] Terminal output or test report showing passing host C++ tests and pytest suite.
- [] Live demonstration of START, STOP, STATUS, REPLAY, and ERASE.
- [] Screenshot showing rolling chart, populated statistics, and file-ring progress during wraparound.
- [] CSV export containing correctly scaled fields.
- [] Design note documenting metadata offsets, recovery logic, and power-loss result.
- [] Pass/fail fault-injection table for the final validation scenarios.

15. Deliverables and Submission

15.1 Required repository content

```
kern-lite/
|-- kern-lite.ioc
|-- Core/ Drivers/ Middlewares/
|-- firmware/
|   |-- hal/ sensors/ dsp/ protocol/
|   |-- storage/ recorder/ system/
|-- groundstation/
|-- tests/
|   |-- host/
|   |-- gs/
|-- docs/
|   |-- README.md
|   |-- design_note.md
|   |-- demo/
|-- .gitignore
```

15.2 Submission artifacts

Artifact	Required content
Firmware source	STM32L4/C++ project using the required layering and building cleanly.
Ground Station	Python application implementing Section 7 and runnable from documented commands.
Tests	Host C++ and pytest tests with one documented command per suite.
README	Build/flash, GS run instructions, pinout, KAT result, constants, test commands, member contributions.
Design note	1-2 pages: exact LogMeta byte layout, recovery strategy, one robustness experiment and result.
Demo session	CSV, alert log, timeline, raw frame log, and wraparound screenshot.
Validation record	Final fault-injection pass/fail table and known limitations.

15.3 Submission rules

- Submit one repository or archive containing firmware, groundstation, tests, docs, and README.
- Do not commit compiler build output or generated session data except the selected demonstration artifacts.
- Commit the .ioc, generated Drivers/Middlewares/Core files needed for reproducible build, and all application source.
- Every group member's contribution shall be summarized briefly in README.md.
- Any instructor-approved deviation from protocol, record, metadata, or pin assignments shall be documented explicitly.
- The final demo shall include live wraparound and reboot-recovery evidence.

16. Grading Considerations

Area	Weight	Evidence that earns marks
Protocol correctness and CRC	20%	Exact frame layout; 9-byte overhead; dual CRC behavior; KAT and cross-vectors; correct ACK/NACK semantics.
Circular file storage	20%	Sequential write, file advance, wrap, overwrite, replay clamp/order, metadata correctness.
Reboot recovery	10%	Stale checkpoint forward scan and full-scan recovery restore the correct head without corrupting prior records.
Firmware architecture and RTOS	15%	Layering, static allocation, bounded ISR, task interfaces, synchronization, fault policy.
Ground Station	20%	Correct decode, integrity, controls, reconnect, charts, statistics, timeline, session, storage model, export.
Unit tests	10%	Meaningful host tests covering happy paths, boundaries, corruption, and recovery logic.
Documentation and demo	5%	Reproducible README, design note, exports, screenshot, live evidence.

ROBUSTNESS BONUS Up to +5% may be awarded for clean power-loss survival, graceful reconnect, strong recovery tests, and correct handling when REPLAY exceeds retained capacity.

APPENDIX A. FIXED CONSTANTS AND DEFAULTS

Constant / setting	Required default
MCU / board	STM32L476RG / NUCLEO-L476RG
RTOS interface	CMSIS-RTOS2 / CMSIS_V2
System clock	80 MHz
UART	USART2, 115200 8N1
STX / ETX	0xAB / 0xCD
Maximum payload	256 bytes
Frame overhead	9 bytes
Maximum encoded frame	265 bytes
Sample period	100 ms
DHT measurement interval	about 2 s
DSP window	16
Log files	4
Records per file	256
Ring capacity	1024 records
Metadata flush	16 records
Erase magic	0xDEADC0DE
Metadata magic	0x4C4F4700
Metadata version	1
Maximum consecutive write failures	3
STATUS heartbeat	5 s
GS heartbeat timeout	15 s
Default replay request	120 records
Record size	32 bytes
Metadata size	36 bytes

APPENDIX B. GLOSSARY

Term	Definition
ACK	Successful command-completion frame.
Circular ring	Fixed-capacity storage that overwrites the oldest retained data after wrapping.
CRC	Cyclic redundancy check used to detect accidental corruption.
Frame CRC	CRC over serial frame TYPE, LEN, and PAYLOAD.
Record CRC	CRC stored inside SensorRecord over its first 28 bytes.
Checkpoint	Persisted metadata that may trail the true write head by a bounded number of records.
Hysteresis	Different entry and exit boundaries that prevent alert chatter.
Last known good	Most recent valid sensor value retained after a sensor read fault.
NACK	Command-rejection frame carrying a reason code.
Retained count	$\min(\text{total_records}, \text{ring capacity})$.
Sequence gap	Unexpected discontinuity in the 16-bit record sequence.
STATUS heartbeat	Periodic valid STATUS frame used to prove protocol liveness.
Storage corruption warning	GS condition where transport CRC is valid but record CRC is invalid.
Write head	The next circular record position that will be written.

DEFINITION OF DONE The system meets the mandatory requirements, all tests pass, the complete command cycle works on hardware, wraparound and recovery are demonstrated, integrity faults are visible, and all required artifacts are submitted.